TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Administración y seguridad de Food Store

Trabajo práctico - Unidad 5 | Base de Datos II

Exequiel Arancibia · Joaquin Romero

Ciclo lectivo 2026 · PostgreSQL 17.11

Roles, auditoría, reconstrucción de incidentes y protección de datos personales. Resolución sobre la base acumulada de los TP1 a TP5 y la Unidad 4.

Este informe acompaña al repositorio entregado como ZIP. Allí se incluyen SQL, herramientas, respuestas completas de IA, resultados de ejecución e historial Git. Las credenciales y los logs originales se conservan fuera del repositorio.

Continuidad del proyecto y alcance de Unidad 5

Base heredada

La resolución utiliza Food Store de Exequiel Arancibia y Joaquin Romero y conserva la continuidad de los trabajos anteriores. El laboratorio U5 utiliza PostgreSQL 17.11. Su copia privada tiene como punto de partida la base acumulativa U4, con las cinco tablas de negocio originales en `public` y estas cantidades anteriores al alta de la cuenta U5:

Tabla	Filas heredadas de U4
`categoria`	2
`producto`	50.000
`usuario`	20.000
`pedido`	225.000
`detalle_pedido`	499.997

La trazabilidad del modelo es la siguiente:

Antecedente	Continuidad conservada
TP1	Tablas, PK/FK, baja lógica, identificadores BIGINT, tipos de estado/pago, precio histórico y subtotal calculado.
TP2	Transiciones de pedido y validación que permite editar detalles solo en pedidos PENDIENTE; recálculo del total.
TP3 y TP4	Carga masiva, respaldo y optimizaciones del proyecto anterior.
TP5	Índices y vistas, incluida la materializada mensual por categoría.
Unidad 4	Descomposición FNBC del control de lotes, su vista compatible, guardia de unicidad y materializada diaria con sincronización transaccional.

Los respaldos de la cadena de reconstrucción se identifican por hashes diferentes:

Respaldo	SHA-256 verificado
Original pre-TP4	`f53af18eac3da8de40114f63daaea8ccddae528db688b68e6740c8e31fc01871`
Copia acumulativa U4 usada para preparar U5	`47e528484609b85dd3076eda5d7c36180218192c024ae00475b0219d4708251e`

Los índices/vistas de TP4-TP5 y las incorporaciones U4 completaron el estado acumulativo desde el cual se preparó la copia U5. El hash identifica cada archivo sin publicar su contenido.

El respaldo privado de entrada U5 permanece en el espacio de ejecución externo al repositorio. No se lo incorpora al repositorio ni al ZIP, ni se publican rutas privadas del usuario. Quien reconstruya necesita disponer privadamente de una copia equivalente, indicarla mediante `--source-dump` y mantenerla fuera de Git.

Alta sintética comprobada

Se ejecutó el alta de una cuenta comercial de laboratorio en la tabla `public.usuario` existente. No se recreó el maestro ni se convirtieron las contraseñas heredadas de demostración en nuevas claves. El alta usa un correo sintético y una clave aleatoria privada; `pgcrypto`, instalado en `crypto`, genera el hash para comprobar autenticaciones de esta cuenta.

Los resultados publicados en continuidad_verificada.json (continuidad_verificada.json) son:

Control	Resultado
Usuarios heredados	20.000
Cuenta sintética agregada	1
Usuarios totales posteriores	20.001
Huella de los usuarios heredados antes	`10d08c7a44644392c7cfd0f68827c270`
Huella de los usuarios heredados después	`10d08c7a44644392c7cfd0f68827c270`

El control ordena los registros y compara una huella MD5 de sus filas completas, excluyendo del conjunto posterior la cuenta agregada. La coincidencia es la evidencia de conservación obtenida en este laboratorio; no constituye una técnica de anonimización. La evidencia pública no contiene filas personales, correo de la cuenta, contraseñas ni hashes de contraseñas.

Parte D inicial y reconstrucción final bajo el propietario correcto

La primera ejecución de D se realizó el 21/10/2026 con el administrador técnico de bootstrap, durante la espera de Kiro. Se conserva como antecedente en d_bootstrap_equivalencia.json (evidencias/d_bootstrap_equivalencia.json) y d_bootstrap_ejecucion_psql.txt (evidencias/d_bootstrap_ejecucion_psql.txt). Ese resultado inicial no se usa para atribuir permisos a roles que todavía no se habían implementado.

Después de aplicar A se repitió D el **22/10/2026 a las 16:46:53, UTC-03**. El SQL final ejecuta `SET LOCAL ROLE rol_propietario` dentro de la transacción; la tabla reconstruida tiene ese propietario y el permiso SELECT efectivo de `rol_reportes` quedó comprobado. d_equivalencia.json (d_equivalencia.json), d_ejecucion_psql.txt (d_ejecucion_psql.txt) y d_resultados_lado_a_lado.md (d_resultados_lado_a_lado.md) acreditan nuevamente **20.001 filas, dos grupos y diferencias 0/0**. La huella de los 20.000 usuarios heredados también permanece intacta.

La migración de propiedad y las concesiones sobre la copia preexistente se trataron explícitamente en A; los defaults no se consideraron retroactivos. La reconstrucción final, con `rol_propietario` como creador real, comprobó que la política futura de `datos_anon` permite SELECT a reportes. Para reproducir el orden de trabajo, la herramienta ofrece `--bootstrap` solo para la preparación anterior a A; después de A se utiliza su modo normal. verificacion_equivalencia.md (verificacion_equivalencia.md) detalla ambos momentos y las huellas del SQL efectivamente ejecutado.

Conservación final de las cinco tablas de negocio

Se compararon conteos y huellas MD5 ordenadas del contenido completo de cada tabla antes de aplicar A y después de las pruebas A/B/C y la reconstrucción final D. La comparación de conservacion_after.json (evidencias/conservacion_after.json) contra conservacion_before.json (evidencias/conservacion_before.json) registra `same_as_before = true`:

Tabla	Filas antes y después	Huella de contenido coincidente
`categoria`	2	Sí
`producto`	50.000	Sí
`usuario`	20.001	Sí
`pedido`	225.000	Sí
`detalle_pedido`	499.997	Sí

Este segundo control incluye la cuenta sintética U5 ya existente y complementa el control separado de los 20.000 usuarios originales. No se exportaron filas reales para obtener las huellas. Las inserciones y actualizaciones de prueba se revirtieron; el rollback no devuelve los valores consumidos de las secuencias identity ni borra los registros de auditoría.

Compatibilidad conservada por la implementación U5

- `pedido.fecha` es TIMESTAMPTZ; los agrupamientos temporales usan la zona local del negocio.

- `detalle_pedido` tiene PK compuesta `(pedido_id, producto_id)`, sin ID sustituto ni columna `eliminado`.

- `subtotal` es GENERATED STORED; la aplicación modifica cantidad/precio unitario, no escribe el subtotal.

- Los pedidos nacen PENDIENTE y las reglas heredadas restringen la edición de sus detalles.

- La extensión FNBC se conserva y queda fuera de los permisos de aplicación de U5.

- Las materializadas mensual y diaria tienen finalidades distintas. El mantenimiento interno de la diaria se realiza mediante un puente SECURITY DEFINER acotado, sin conceder MAINTAIN a la aplicación ni escritura sobre totales derivados.

La especificación real de Kiro distingue cuatro grupos operativos, dos logins y dos roles técnicos NOLOGIN. Sus documentos finales y respuestas exportadas están en kiro_roles_spec (kiro_roles_spec/estado.md), con la revisión automática de Codex identificada por separado. El commit `319cbaa` conserva esa especificación antes del commit `4262d81`, que incorpora `roles.sql`; no se presenta el contexto de entrada como si fuera una salida de Kiro.

La verificación efectiva de A finalizó con 52 eventos en permissions_resultados.json (evidencias/permissions_resultados.json) y 71 eventos en defaults_resultados.json (evidencias/defaults_resultados.json), ambas fases con estado `passed`. La comparación exacta de ACL identificó **44 concesiones explícitas de negocio previstas y 44 reales**, sin faltantes, sobrantes ni diferencias de membresías, según a_comparacion_exacta.json (evidencias/a_comparacion_exacta.json). Estos conteos son eventos de los verificadores y entradas de ACL; no se los confunde con una validación humana.

Alcance de seguridad y auditoría

U5 demuestra concesiones puntuales por objeto/columna, funciones de autenticación y restablecimiento, permisos futuros bajo el creador correcto, eventos de auditoría verificables, un escalamiento rechazado y agregaciones equivalentes sobre una copia desidentificada. El simulacro se limitó al clúster propio de desarrollo. Después de recibir el resumen A–E y la propuesta de revisión, la persona solicitante dio conformidad al cierre y pidió el ZIP para subirlo a la plataforma. El alcance de esa conformidad se registra en validacion_humana.md (validacion_humana.md): no acredita lectura exhaustiva ni aprobación individual de ambos integrantes. No se ejecutó contención automática a partir de una respuesta de IA.

El laboratorio se configuró con SCRAM y escucha exclusiva en `127.0.0.1`; conserva secretos y logs en un runtime externo con ACL restringida. Durante la preparación se desactivó el registro de sentencias; antes de B/C se verificaron conexiones y desconexiones activadas, `log_statement=all`, `log_min_error_statement=error` y ambos límites de parámetros en cero. Los valores efectivos quedaron registrados en configuracion_auditoria.json (configuracion_auditoria.json).

La prueba B completó 10 eventos y comprobó lectura, inserción, actualización, rollback y una conexión PostgreSQL rechazada por contraseña incorrecta. El simulacro C completó 11 eventos: tres autenticaciones comerciales fallidas, una exitosa, recepción de 20.001 filas autorizadas y escalamiento rechazado con SQLSTATE `42501`, sin nueva membresía. Los resultados están en audit_resultados.json (evidencias/audit_resultados.json) e incident_resultados.json (evidencias/incident_resultados.json). Las autenticaciones comerciales y la autenticación de la conexión PostgreSQL son controles diferentes.

`log_statement=all` permite registrar sentencias y metadatos; no equivale a un historial de los valores OLD/NEW de cada fila ni demuestra por sí solo cuántos registros recibió el cliente. En C, el conteo recibido se comprobó en el cliente y no se dedujo solamente del SELECT registrado. La documentación oficial describe los niveles de sentencias y el tratamiento de parámetros. PostgreSQL 17: registro de errores y actividad (https://www.postgresql.org/docs/17/runtime-config-logging.html).

pgAudit puede agregar auditoría por sesión, clases de operación y objetos; su opción `log_rows` registra un número de filas, no los valores OLD/NEW de cada una. No se le atribuye automáticamente un historial fila por fila. Obtener ese historial requeriría otro mecanismo diseñado para ello. No se afirma que pgAudit esté instalado en este laboratorio. pgAudit para PostgreSQL 17 (https://github.com/pgaudit/pgaudit/blob/REL_17_STABLE/README.md).

Los logs son activos sensibles: pueden revelar identidades, orígenes, estructura y operaciones aun cuando los parámetros estén suprimidos. Los originales permanecen privados; para prompts y evidencias públicas se usa una copia anonimizada con controles técnicos de privacidad. No se publican credenciales ni filas personales. La copia determinista de usuarios se describe con sus límites de reidentificación, sin afirmar irreversibilidad universal.

Estado honesto y forma de entrega

La especificación real de Kiro, la implementación A y las ejecuciones técnicas A/B/C/D tienen evidencias verificables. Las cinco tablas de negocio y los 20.000 usuarios originales conservan sus huellas de contenido. La persona solicitante autorizó el cierre con el pedido literal «bien entonces enviame el zip listo para entregar asi lo subo a la plataforma». Esa conformidad se distingue de la revisión automática de Codex y tiene el alcance documentado en validacion_humana.md (validacion_humana.md), sin atribuir una revisión exhaustiva ni la validación separada de ambos autores.

El servidor privado de PostgreSQL quedó detenido mediante `pg_ctl` con modo `fast` y espera de finalización: el comando terminó con código 0 y confirmó `server stopped`. Sus archivos se conservan para una eventual reanudación; no se borró el clúster.

La entrega se prepara como repositorio Git, o su ZIP con `.git` e historial legible por partes, más un PDF complementario. El respaldo, el runtime, las claves y los logs sin anonimizar quedan excluidos. El cierre y la verificación del paquete están en proceso; no se afirma que la entrega ya haya sido publicada o enviada.

Parte A. Roles y permisos de Food Store

Exequiel Arancibia — Joaquin Romero. Base de Datos II, Unidad 5, 2026.

Estado: implementación instalada y verificación técnica de A ejecutada. La comparación exacta coincide en 44 entradas ACL; las fases permissions y defaults finalizaron correctamente con 52 y 71 eventos registrados. Se adjuntan capturas reales y auditoría OpenCode. El flujo positivo de pedido/detalle, la reconstrucción D y la conservación posterior también quedaron comprobados por sus evidencias separadas. La persona solicitante dio conformidad al cierre y autorizó preparar el ZIP para entregar, con el alcance registrado en validacion_humana.md (validacion_humana.md). Esto no acredita lectura exhaustiva ni aprobación individual de ambos integrantes; las verificaciones y correcciones técnicas siguen identificadas como Codex.

A.1. Planificación previa en Kiro

Primero se inventarió la base propia de desarrollo, continuando los trabajos anteriores: tablas, claves, columnas GENERATED/IDENTITY, vistas, MVs y triggers de TP2/U4. La copia de trabajo contiene 20.000 usuarios heredados y una cuenta sintética de prueba; el dump con datos y sus secretos permanecen fuera del repositorio. Se reconstruyeron los artefactos de roles y las funciones de autenticación/reset para este entorno, dejando su procedencia identificada, en lugar de atribuirles una existencia previa no comprobada.

Después se proporcionó ese inventario a Kiro y se revisaron sus propuestas. Las respuestas reales finales de requirements y design se exportaron desde la aplicación y se preservaron con sus marcas y hashes. El commit `319cbaa` guardó ambos documentos y la revisión automática de Codex **antes** de crear `sql/roles.sql`.

Rol	Tipo	Operación habilitada y justificación
rol_app_lectura	NOLOGIN	Leer catálogo, pedidos, detalle y una proyección de usuario, porque la aplicación necesita consultar el negocio sin acceder a hashes.
rol_app_escritura	NOLOGIN	Escribir columnas operativas de pedido/detalle, porque los importes derivados y las credenciales tienen mecanismos específicos.
rol_soporte	NOLOGIN	Consultar contacto y cambiar teléfono, porque la atención al cliente no necesita pedidos, catálogo o contraseñas.
rol_reportes	NOLOGIN	Leer dos agregados materializados y usuario_anon, porque los informes no requieren datos personales fuente.
rol_propietario	NOLOGIN	Poseer infraestructura e invariantes heredadas, porque los logins de uso cotidiano no deben controlar objetos ni permisos.
rol_funciones	NOLOGIN	Poseer sólo cuatro funciones protegidas, porque ciertos cálculos y credenciales requieren privilegios internos limitados.
app_web	LOGIN	Heredar lectura/escritura y ejecutar autenticación, porque representa la conexión de la aplicación.

Rol	Tipo	Operación habilitada y justificación
admin_datos	LOGIN	Heredar soporte y ejecutar reset, porque la administración de contacto/credenciales requiere una cuenta diferenciada.

Las únicas membresías son app_web → lectura/escritura y admin_datos → soporte. Las tres tienen INHERIT TRUE, SET FALSE y ADMIN FALSE: heredan las ACL de sus grupos, no habilitan SET ROLE hacia ellos ni permiten conceder membresías. Ningún login es miembro de los dos propietarios técnicos. Ninguno de los ocho roles es superusuario, creador de bases/roles, replicador ni BYPASSRLS.

Los documentos reales se conservan en requirements (kiro_roles_spec/requirements.md), design (kiro_roles_spec/design.md) y revisión automática (kiro_roles_spec/revision_automatica_codex.md). La matriz final más estrecha de soporte tiene seis columnas, sin created_at. La ampliación preliminar de Kiro a pedidos/detalles fue rechazada y consta en su historial; se distingue de la auditoría OpenCode del punto A.4.

A.2. Implementación de CREATE ROLE, REVOKE y GRANT

El archivo roles.sql (../sql/roles.sql) comienza con una transacción y precondiciones: PostgreSQL 17, base y bootstrap esperados, ausencia de los ocho roles y existencia de los objetos heredados/pgcrypto. Si hay un rol U5 preexistente, aborta; no lo reemplaza silenciosamente. No contiene claves. Los logins nacen con PASSWORD NULL y el instalador configura después sus secretos privados.

La implementación sigue estos pasos:

1. Crea seis roles NOLOGIN y dos LOGIN; aplica sólo las tres membresías.

2. Transfiere esquemas, tablas, vistas, MVs, secuencias y enums a

rol_propietario. La transferencia de las tablas mueve sus secuencias IDENTITY dependientes; no se intenta cambiarles dueño por separado.

3. Revoca permisos heredados de PUBLIC y limpia tanto ACL de tabla como

de columna. Revocar una tabla no elimina por sí solo sus GRANT por columna.

4. Protege los dos puentes heredados y crea autenticación/reset con

SECURITY DEFINER, dueño rol_funciones, search_path fijo y objetos calificados. Conserva las demás funciones como INVOKER.

5. Concede exactamente los permisos de la matriz, USAGE de esquemas donde

hace falta y CONNECT a los dos logins. Quita CREATE/TEMP de PUBLIC; TEMP se reserva al propietario técnico para la auditoría de equivalencia D.

6. Establece defaults para los creadores efectivos y deja la comprobación

de objetos futuros en transacciones reversibles.

Beneficiario	Objeto	Privilegio y columnas
rol_app_lectura	categoria, producto, pedido, detalle_pedido	SELECT de tabla.
rol_app_lectura	usuario	SELECT(id,nombre,apellido,email,telefono,eliminado,created_at).

Beneficiario	Objeto	Privilegio y columnas
rol_app_escritura	pedido	INSERT(usuario_id,fecha,forma_pago); UPDATE(estado,forma_pago,eliminado).
rol_app_escritura	detalle_pedido	INSERT(pedido_id,producto_id,cantidad,precio_unitario); UPDATE(cantidad,precio_unitario); DELETE sujeto al trigger de PENDIENTE.
rol_soporte	usuario	SELECT(id,nombre,apellido,email,telefono,eliminado); UPDATE(telefono).
rol_reportes	datos_anon.usuario_anon	SELECT.
rol_reportes	mv_facturacion_categoria_mes, mv_u4_ventas_categoria_dia	SELECT.
app_web	fn_autenticar(text,text)	EXECUTE directo.
admin_datos	fn_resetear_password(bigint,text)	EXECUTE directo.
rol_funciones	usuario	SELECT(id,email,contrasena,eliminado); UPDATE(contrasena).
rol_funciones	detalle_pedido	SELECT(pedido_id,subtotal).
rol_funciones	pedido	SELECT(id); UPDATE(total).
rol_funciones	mv_u4_ventas_categoria_dia	Sólo MAINTAIN.

El caso de permiso por columna es explícito. Por ejemplo:

```
GRANT SELECT (id,nombre,apellido,email,telefono,eliminado,created_at)
    ON public.usuario TO rol_app_lectura;
GRANT SELECT (id,nombre,apellido,email,telefono,eliminado), UPDATE (telefono)
    ON public.usuario TO rol_soporte;
```

No existe SELECT de tabla sobre usuario para esos roles, por lo que un SELECT * que incluya contrasena queda denegado. Tampoco se conceden DELETE de pedido/usuario, escritura de total/subtotal, cambios de catálogo, MAINTAIN a logins, acceso a secuencias ni privilegios sobre la extensión FNBC.

Los cuatro contratos protegidos son:

Función	Resultado y restricción
fn_autenticar(text,text)	id BIGINT para credenciales bcrypt válidas de usuario activo; NULL si inválidas/eliminado. No acepta los hashes demostrativos como texto plano. Registra sólo resultado y timestamp.
fn_resetear_password(bigint,text)	Retorna void. Sólo admin_datos; longitud 12..72 bytes o SQLSTATE 22023; usuario inexistente o id NULL, P0002. Bcrypt nuevo con coste 10; mensajes sin credenciales.
recalcular_total_pedido()	Conserva el recálculo de TP2 mediante UPDATE total reservado a la función; la aplicación no recibe ese UPDATE.
u4_refrescar_ventas()	Refresca sólo la MV diaria con MAINTAIN interno; conserva la omisión del UPDATE anidado de pedido.

Todas tienen search_path=pg_catalog y REVOKE EXECUTE FROM PUBLIC. Los auxiliares de pgcrypto sólo se conceden a rol_funciones. Autenticar no cambia current_user ni otorga permisos SQL a la identidad del negocio devuelta. Este diseño no implementa aislamiento por fila mediante RLS.

La política de objetos futuros es igualmente explícita:

```
ALTER DEFAULT PRIVILEGES FOR ROLE rol_propietario IN SCHEMA datos_anon
    GRANT SELECT ON TABLES TO rol_reportes;
```

En public no se concede acceso automático futuro. El SQL limpia defaults de tablas/secuencias y revoca EXECUTE público por defecto de funciones de forma global para rol_propietario, rol_funciones y u5_bootstrap. Un REVOKE de funciones limitado a un esquema no revoca el default global de PUBLIC. Los defaults no son retroactivos: usuario_anon existente recibe propietario y GRANT explícitos; su reconstrucción D debe usar el creador correcto.

Los contratos y supuestos completos están en a_contratos_y_supuestos.md (a_contratos_y_supuestos.md).

A.3. Verificación ejecutada y evidencia

La instalación se ejecutó en la copia privada y después se capturó su estado real. El 22/10/2026 a las 19:47:13 UTC, la comparación exacta arrojó 44 ACL previstas y 44 actuales, cero faltantes y cero adicionales; también cero membresías faltantes o adicionales. Esto verifica la matriz de tablas/columnas y membresías, no constituye una certificación universal de seguridad. Los privilegios implícitos del propietario se excluyen de esa comparación y permanecen documentados.

El punto pide \du y information_schema.role_table_grants. El capturador conserva ambas salidas y amplía su alcance a privilegios por columna, propietarios, funciones, default ACL y catálogos de PostgreSQL 17. Esta ampliación importa porque role_table_grants por sí sola no demuestra la proyección de usuario ni el MAINTAIN de una MV.

```
-- En psql, como verificador técnico:
\du
\ddp
SELECT grantee,table_schema,table_name,privilege_type,is_grantable
FROM information_schema.role_table_grants
WHERE table_schema IN ('public','datos_anon')
  AND grantee IN ('rol_app_lectura','rol_app_escritura','rol_soporte',
                  'rol_reportes','rol_funciones','app_web','admin_datos')
ORDER BY grantee,table_schema,table_name,privilege_type;
SELECT grantee,table_schema,table_name,column_name,privilege_type,is_grantable
FROM information_schema.role_column_grants
WHERE table_schema IN ('public','datos_anon')
  AND grantee IN ('rol_app_lectura','rol_app_escritura','rol_soporte',
                  'rol_reportes','rol_funciones','app_web','admin_datos')
ORDER BY grantee,table_schema,table_name,column_name,privilege_type;
```

Comandos técnicos empleados, desde la raíz del repositorio y con U5_SECRET_FILE configurado hacia el JSON privado externo:

```
python tools/instalar_seguridad.py
python tools/capturar_permisos.py
python tools/verificar_matriz.py
python tools/ejecutar_pruebas.py --phase permissions
python tools/ejecutar_pruebas.py --phase defaults
```

La comparación exacta usa ACL de tabla y columna por separado y excluye privilegios implícitos del dueño. Comprueba también las tres membresías y sus opciones. La captura complementaria inspecciona schemas/base, propietarios, funciones y defaults; las pruebas de comportamiento usan conexiones reales app_web/admin_datos y distinguen los grupos probados mediante SET LOCAL ROLE con bootstrap.

Evidencia real	Resultado observado
a_psql_du.txt (evidencias/a_psql_du.txt) y a_psql_ddp.txt (evidencias/a_psql_ddp.txt)	Ocho roles U5 y cuatro filas de defaults: tres funciones globales sin PUBLIC EXECUTE y SELECT de reportes en datos_anon para el creador propietario.
a_permisos.json (evidencias/a_permisos.json) y archivos TSV	Captura real de atributos, tablas/columnas, funciones, propietarios, membresías y esquemas/base.
a_comparacion_exacta.json (evidencias/a_comparacion_exacta.json)	PASSED; 44/44 ACL, diferencias 0/0 y diferencias de membresías 0/0.
permissions_resultados.json (evidencias/permissions_resultados.json)	PASSED; 52 eventos entre 19:44:39.873 y 19:44:40.380 UTC.
defaults_resultados.json (evidencias/defaults_resultados.json)	PASSED; 71 eventos entre 19:44:48.327 y 19:44:48.452 UTC; sin objetos de prueba remanentes.
audit_resultados.json (evidencias/audit_resultados.json)	PASSED; 10 eventos: lectura, inserción de pedido/detalle, actualización, total correcto, transición y ROLLBACK; también login PostgreSQL fallido.
incident_resultados.json (evidencias/incident_resultados.json)	PASSED; 11 eventos: tres autenticaciones fallidas y una válida; lectura autorizada de 20.001 filas sin exportarlas; GRANT denegado con 42501 y membresía administrativa ausente antes/después.
d_equivalencia.json (d_equivalencia.json)	Reconstrucción bajo rol_propietario: 20.001 filas, dos grupos y diferencias EXCEPT ALL 0/0; reportes tiene SELECT automático.
conservacion_before.json (evidencias/conservacion_before.json) y conservacion_after.json (evidencias/conservacion_after.json)	Mismos conteos y huellas de contenido de las cinco tablas; same_as_before=true.

El detalle de la comprobación permite distinguir lo siguiente:

- **Atributos y membresías:** únicamente app_web/admin_datos pueden iniciar

sesión entre los ocho roles U5; los cinco atributos peligrosos están en falso para los ocho. Existen exactamente las tres membresías previstas, con ADMIN=false, INHERIT=true y SET=false. Ver atributos (evidencias/a_atributos.tsv) y membresías (evidencias/a_membresias.tsv).

- **Funciones:** exactamente fn_autenticar, fn_resetear_password,

recalcular_total_pedido y u4_refrescar_ventas pertenecen a rol_funciones, tienen prosecdef=true y search_path=pg_catalog. Las otras seis funciones públicas pertenecen a rol_propietario y son INVOKER. La ACL autoriza autenticación sólo a app_web y reset sólo a admin_datos, además de su dueño; ningún EXECUTE público consta en esas funciones. Ver funciones (evidencias/a_funciones.tsv).

- **Propiedad:** las 22 relaciones inventariadas de public/datos_anon

—tablas, cuatro secuencias, vistas y dos MVs— pertenecen a rol_propietario; rol_funciones no posee ninguna. Ver propietarios (evidencias/a_propietarios.tsv).

- **Esquemas/base:** los roles de negocio no tienen CREATE en los tres

esquemas ni CREATE/TEMP de base. Los logins tienen CONNECT y USAGE de public, no de crypto/datos_anon. rol_funciones usa public/crypto sin CREATE; rol_reportes usa public/datos_anon sin CREATE. Sólo rol_propietario tiene TEMP entre los ocho roles. Ver esquemas (evidencias/a_esquemas_efectivos.tsv) y base (evidencias/a_base_efectivos.tsv).

- **Comportamiento:** app_web obtuvo SQLSTATE 42501 al leer hashes o

SELECT * de usuario, escribir contrasena/total, eliminar pedido/usuario, asumir roles no habilitados, crear roles, usar nextval/setval o ejecutar reset. admin_datos leyó exactamente seis columnas y pudo ejecutar reset; se rechazaron created_at, hashes, DELETE de usuario, lectura de pedido y actualización de columnas ajenas a teléfono. El UPDATE de contacto se probó con WHERE FALSE: demuestra autorización, no modificación de una fila.

- **Contrato de reset:** usuario inexistente produjo P0002; NULL y

longitudes 11/73 bytes y multibyte de 74 bytes produjeron 22023. Los límites válidos de 12 y 72 bytes se aceptaron. Los cambios se revirtieron.

- **Defaults:** una tabla nueva en public negó SELECT/INSERT/UPDATE/DELETE

a los seis roles de negocio. En datos_anon, sólo reportes recibió SELECT. Las funciones creadas realmente por bootstrap, propietario y funciones tuvieron cero entradas PUBLIC EXECUTE y denegaron ejecución con 42501 a los seis roles. El CREATE concedido a rol_funciones sólo para esa prueba quedó dentro de la transacción: después del ROLLBACK volvió a false y no quedaron funciones ni tablas de prueba.

- **Puentes heredados en uso:** el flujo real app_web de la Parte B insertó

un pedido PENDIENTE y su detalle, actualizó cantidad y comprobó total igual a la suma de subtotales; luego confirmó el pedido y revirtió las filas. El camino con triggers U4 completó correctamente sin conceder UPDATE total ni MAINTAIN a app_web. Esta prueba no se presenta como una comparación exhaustiva del contenido de ambas MVs.

- **Conservación y creador correcto:** la reconstrucción real de

usuario_anon bajo rol_propietario obtuvo SELECT para reportes por default y equivalencia bilateral 0/0. La captura posterior conserva exactamente 2 categorías, 50.000 productos, 20.001 usuarios, 225.000 pedidos y 499.997 detalles, con iguales huellas completas a las anteriores. Los 20.000 usuarios originales y la cuenta sintética de U5 permanecen intactos.

Los cambios de prueba se revierten; los valores consumidos por IDENTITY no retroceden con ROLLBACK. Las huellas previas se calcularon sobre filas completas ordenadas sin exportar las filas fuente. Ninguna prueba debe ampliar la matriz para conseguir un resultado positivo.

A.4. Auditoría de OpenCode y lectura crítica

Se solicitó una revisión estática real a OpenCode 1.18.23, modelo openai/gpt-6.1-sol, agente plan y permisos de herramientas denegados. La consulta terminó el 22/10/2026 a las 19:43:21 UTC sin ejecutar SQL ni otras herramientas. Se conservan el prompt (ia/a_revision_prompt.txt), los metadatos (ia/a_revision_metadatos.json), los adjuntos exactos y la respuesta completa (ia/a_revision_respuesta.md).

El auditor no identificó un incumplimiento bloqueante demostrado de la matriz final. Señaló tres condiciones/riesgos: los literales SQL pueden filtrar credenciales aunque se oculten parámetros; la limpieza no elimina ACL de cualquier rol ajeno preexistente; y la autenticación conserva diferencias temporales y no limita intentos. Se aceptó técnicamente esa distinción: se verificaron clientes parametrizados y límites de logging, se compararon las ACL reales del laboratorio y se dejaron documentadas las limitaciones de autenticación sin inventar mitigaciones implementadas.

El prompt pidió además alternativas operativas para evaluarlas críticamente. OpenCode propuso dos candidatos más amplios, advirtiendo expresamente que excedían la matriz y recomendando conservar los GRANT puntuales:

Propuesta real solicitada	Acceso adicional	Decisión técnica de Codex
Heredar rol_app_lectura desde rol_soporte	Soporte/admin_datos leerían catálogo, pedidos, detalles y usuario.created_at, además de futuras concesiones del grupo.	Rechazada: soporte sólo requiere seis columnas de contacto y UPDATE telefono. La captura conserva exactamente tres membresías, sin esta ampliación.
GRANT SELECT ON ALL TABLES de public a reportes y el default equivalente	Reportes leería usuario.contrasena, contacto real, fuentes del negocio, FNBC y futuros objetos públicos del creador indicado.	Rechazada: se mantienen sólo usuario_anon y las dos MVs. La comparación exacta no detectó ACL adicionales.

Son alternativas solicitadas y candidatas inseguras para este alcance; **no se presentan como errores espontáneos de OpenCode**. El auditor ya explicó sus excesos. La decisión técnica fue no aplicarlas, demostrada por la matriz actual. La conformidad de cierre de la persona solicitante consta con su alcance en validacion_humana.md (validacion_humana.md), sin atribuir una aprobación individual a ambos integrantes. El contraste puntual está en a_revision_validacion.md (ia/a_revision_validacion.md); el registro correspondiente debe conservarse también en BITACORA.md.

Parte B. Configuración y prueba de auditoría de Food Store

Ejecución técnica verificada sobre PostgreSQL 17.11 el 22/10/2026.

La prueba B se ejecutó entre 19:45:41.715 y 19:45:43.743 UTC (16:45:41.715–16:45:43.743, UTC−03). El cliente registró diez comprobaciones y el fragmento nativo conserva catorce eventos de dos sesiones completas. La revisión técnica y de privacidad aquí documentada es automática, realizada con Codex. El alcance de la conformidad de cierre expresada por el usuario se documenta en validacion_humana.md (validacion_humana.md). La propuesta real de OpenCode y sus correcciones técnicas están conservadas en `ia/bc_respuesta.md` e `ia/bc_validacion_tecnica.md`.

Dependencia o comprobación	Evidencia real
Parte A: especificación final de Kiro y contraste exacto de permisos	`evidencias/a_comparacion_exacta.json`: aprobado, 44 ACL explícitas previstas y observadas; sin faltantes ni extras
Parámetros efectivos en una conexión nueva	`configuracion_auditoria.json`: verificados a las 16:45:36.397328−03
Lectura, inserción y actualización como `app_web`	`evidencias/audit_resultados.json`: estado `passed`, diez eventos cliente
Conexión con contraseña PostgreSQL incorrecta	FATAL real `28P01`, sesión `6ada67e7.5e60`
Captura, anonimización e integridad del fragmento	`log_auditoria_prueba.txt`: 14 JSON válidos; hashes y selección en `evidencias/logs_procedencia.json`

1. Parámetros efectivos y su finalidad

El registro de sentencias estuvo desactivado durante la preparación de credenciales. La conexión de verificación posterior a su activación observó los siguientes valores para las pruebas B/C.

Parámetro	Valor observado	Uso en el laboratorio
`logging_collector`	`on`	Recopilar archivos del servidor
`log_destination`	`stderr,jsonlog`	Formato legible y formato estructurado
`log_connections`	`on`	Registrar intentos y autorización de conexiones
`log_disconnections`	`on`	Registrar cierre de sesiones
`log_statement`	`all`	Incluir lectura y modificaciones
`log_min_error_statement`	`error`	Conservar la sentencia asociada al error
`log_parameter_max_length`	`0`	Omitir valores de parámetros en sentencias
`log_parameter_max_length_on_error`	`0`	Omitir valores de parámetros en errores
`log_timezone`	`America/Buenos_Aires`	Interpretar la hora del registro

Prefijo efectivo para `stderr`:

```
%m pid=%p session=%c user=%u db=%d app=%a client=%r sqlstate=%e
```

El prefijo relaciona hora, PID, usuario, base, aplicación, cliente, SQLSTATE y sesión. JSON proporciona campos equivalentes, además de `line_num`; los valores nulos pueden omitirse. `log_statement=all` amplía el mínimo de modificaciones de la consigna para poder registrar también el SELECT de C. Los límites de parámetros no ocultan contraseñas escritas como literales SQL. PostgreSQL 17: registro de eventos (https://www.postgresql.org/docs/17/runtime-config-logging.html).

2. Resolución paso a paso de la prueba

1. **Verificar la precondición de permisos.** Contrastar los roles y sus privilegios

con la especificación final de Kiro. `app_web` debe poder realizar las operaciones concretas del catálogo de prueba sin recibir permisos administrativos ni acceso a hashes de contraseñas. Las salidas reales de permisos y la comparación exacta de la Parte A se conservaron en `docs/evidencias/a_*`.

2. **Activar la configuración de auditoría.** El archivo

`sql/configurar_auditoria.sql` contiene los cambios de parámetros recargables. `logging_collector` y los destinos se preparan al crear el clúster; si fuera necesario cambiarlos, debe respetarse su requisito de reinicio. La herramienta `tools/activar_auditoria.py` verifica los parámetros en una conexión nueva y conservó la salida en `docs/configuracion_auditoria.json`.

3. **Comprobar la privacidad antes de enviar credenciales.** El cliente verifica

primero que ambos límites de parámetros sean cero. Los valores sensibles se toman de un archivo privado externo al repositorio y se enlazan con `PQexecParams`; no se incorporan como literales al SQL ni se imprimen.

4. **Ejecutar las operaciones como una conexión real de** `app_web`**.** El catálogo

`sql/auditoria_test.sql` se ejecuta con `python tools/ejecutar_pruebas.py --phase audit`. Es un catálogo de sentencias con marcadores y parámetros, por lo que no corresponde ejecutar el archivo completo directamente en psql.

Dentro de `BEGIN`, el cliente lee productos activos con stock para dos unidades, elige uno y conserva su precio solo en memoria. Inserta un pedido, comprueba mediante `RETURNING` que existe una única fila inicialmente `PENDIENTE`, agrega un detalle de una unidad y actualiza su cantidad a dos. Luego verifica que el total del pedido coincida con la suma de subtotales y confirma el pedido mediante una transición condicionada a `PENDIENTE`.

5. **Revertir las filas de prueba.** El cliente ejecuta `ROLLBACK` también ante un

error y compara los conteos de usuarios, pedidos y detalles antes y después. La prueba comprobó las filas devueltas y los valores esperados; un comando sin error no se contará por sí solo como una fila modificada. El rollback no recupera identidades consumidas ni elimina el registro de las operaciones intentadas.

6. **Probar el inicio de sesión PostgreSQL fallido.** En otra conexión se utiliza

la identidad `app_web` con una contraseña deliberadamente incorrecta. El cliente debe distinguir un rechazo de contraseña de un problema de red; el código SQLSTATE `28P01` se comprobó en el evento real del servidor, porque un fallo durante el establecimiento de la conexión no entrega un `PGresult` al cliente. No se dará el intento por demostrado si solo falla la conexión por una causa distinta.

7. **Capturar y preparar la evidencia.** Se conservó el JSON nativo completo de

las sesiones de prueba, incluido el intento fallido, en el área privada. Como este último evento no contiene `application_name`, se lo identificó por su SQLSTATE, usuario solicitado, hora, origen y `session_id`, sin inventar una etiqueta de aplicación. Se extrajeron las sesiones completas, se anonimizaron identificadores y

se verificó el SHA-256 contra los bytes del archivo entregado. El original privado queda fuera de Git.

La evidencia cliente está en `docs/evidencias/audit_resultados.json` y el fragmento anonimizado en `docs/log_auditoria_prueba.txt`.

3. Cómo responder quién, qué, cuándo y desde dónde

La sesión normal es `6ada67e6.80d4` (PID 32980, origen `127.0.0.1:60562`); el intento fallido corresponde a `6ada67e7.5e60` (PID 24160, origen `127.0.0.1:60563`). Las marcas de tiempo siguientes conservan UTC−03.

Pregunta	Evidencia del fragmento	Interpretación y límite
¿Quién?	`user=app_web`; sesión normal, eventos 2–3: SCRAM y autorización	Rol autenticado del motor; en el FATAL solo consta el usuario solicitado
¿Qué?	Sesión normal: 5 SELECT, 6–7 INSERT, 8 y 10 UPDATE, 11 ROLLBACK	Operaciones solicitadas; las filas y el total se acreditan además por el cliente
¿Cuándo?	Lectura 16:45:42.082; INSERT 16:45:42.087; UPDATE 16:45:42.914; FATAL 16:45:43.743	Hora del servidor, convertida con UTC−03 al contrastarla con el cliente
¿Desde dónde?	`remote_host=127.0.0.1`, puertos 60562 y 60563	Conexiones locales; no hay evidencia de un origen remoto externo
¿Qué conexión?	Sesión normal con `application_name=u5_audit_test`; intento fallido sin ese campo	PID y `session_id` preservan la correlación aun cuando falta la aplicación

La numeración indicada es `line_num` **dentro de cada sesión**. El FATAL es el evento 2 de la sesión fallida y la línea 14 del archivo entregado; su código es `28P01` y el mensaje indica rechazo de contraseña de `app_web`. Los eventos 1–12 de la sesión normal incluyen desde la recepción de conexión hasta su desconexión.

Una sentencia registrada no demuestra por sí sola que se haya confirmado la transacción ni cuántas filas recibió el cliente. Para B, los resultados de `RETURNING`, la comprobación del total y el rollback se contrastaron con la evidencia del cliente; no se infirieron solamente a partir del texto SQL.

4. Autenticación del motor y autenticación de Food Store

La conexión de PostgreSQL identifica al rol de servicio, por ejemplo `app_web`; la función `fn_autenticar` valida por separado una cuenta de Food Store. La prueba de contraseña PostgreSQL incorrecta de B y los intentos de contraseña de negocio de C son sucesos diferentes. La primera corresponde al protocolo de conexión del motor, aquí preparado con SCRAM. PostgreSQL 17: autenticación por contraseña (https://www.postgresql.org/docs/17/auth-password.html).

Invocar `fn_autenticar` correctamente como SQL no implica aceptar una contraseña de negocio: la función puede devolver NULL. En C se observaron cuatro mensajes `U5_AUTH success=f/f/f/t`, sin correo, contraseña ni identificador de usuario, conservados en `log_simulacro_anonimizado.txt`. Esos resultados de negocio no se etiquetan artificialmente como `28P01`: el FATAL de B corresponde a otra conexión.

5. Alcance nativo y nociones de pgAudit

La auditoría nativa preparada permite relacionar conexiones, sentencias y errores, pero no proporciona automáticamente un historial de los valores anteriores y posteriores de cada fila ni identifica a la persona detrás de una cuenta de servicio compartida. Para un historial de cambios de valores se necesitaría un mecanismo adicional diseñado para ese fin, como disparadores de auditoría protegidos o captura de cambios.

pgAudit agrega auditoría de sesión y de objetos, permite identificar relaciones consultadas o modificadas y puede incluir cantidades de filas mediante `pgaudit.log_rows`. Ese contador no constituye una colección de los valores `OLD` y `NEW` de todas las filas: la referencia de la consigna a «detalle fila por fila» debe entenderse con esa precisión. Esta parte describe nociones; no afirma que la extensión esté instalada ni configurada en el laboratorio. Documentación oficial de pgAudit para PostgreSQL 17 (https://github.com/pgaudit/pgaudit/blob/REL_17_STABLE/README.md).

6. ¿El log es un activo sensible? Respuesta en tres oraciones

Sí: el log es un activo sensible porque puede revelar consultas, identidades, horarios, estructura de la base y secretos incluidos accidentalmente en el SQL. Deberían poder leerlo únicamente administradores y auditores autorizados, con lectura restringida y escritura reservada al servicio y a quienes gestionan su retención, mediante ACL de Windows en este laboratorio. Los roles de aplicación, soporte e informes no deben recibir acceso general a archivos del servidor ni `pg_read_server_files` para leerlo, ya que ese rol habilita un alcance mucho mayor que el fragmento de auditoría necesario. PostgreSQL 17: roles predefinidos (https://www.postgresql.org/docs/17/predefined-roles.html).

7. Resultados y procedencia de la evidencia

El cliente recibió tres productos del catálogo, insertó un pedido y un detalle, actualizó una línea y confirmó un pedido inicialmente pendiente. La comprobación del total fue verdadera y el rollback dejó los mismos conteos: 20.001 usuarios, 225.000 pedidos y 499.997 detalles. No se exportaron los valores de las filas ni se afirmó recuperar las identidades consumidas.

La captura partió del archivo JSON cerrado `u5-20261022-121336.json`, de 217 eventos. Se seleccionaron por ventana temporal y aplicación las sesiones reales de B, agregando la sesión del FATAL; luego se conservaron todos sus eventos en el orden original, incluidos los anteriores a la aparición de `application_name`. No se incorporaron sesiones verificadoras ni eventos de C al fragmento de B.

Elemento	SHA-256 de sus bytes
Fuente privada completa, excluida de Git	`de25e590ce9afdc75fef85cd19912e51463af9aae3981ee1df40a86916e45f9d`
Fragmento B anonimizado entregado	`9232abae2d5499f4a70e1997e701e81c0dfa55fbd36cb4ac2c7477dbe291b8d5`

Se sustituyó el nombre de la base por `foodstore_lab` y el usuario de las rutas Windows por `USR_LOCAL`. Los escaneos no detectaron secretos del inventario privado, hashes de credenciales, correos ni contraseñas SQL literales en la selección; la inspección automática con Codex tampoco identificó valores de usuarios en el archivo entregado. Se preservaron los campos de tiempo, rol, sesión, PID, origen y errores y se comprobó que los catorce objetos JSON siguen siendo válidos. Estos controles reducen exposición y verifican la transformación; no certifican anonimización irreversible ni autenticidad forense del origen.

Trazabilidad: `evidencias/logs_procedencia.json`; escaneo: `evidencias/b_anonimizacion_scan.json`. La conformidad de cierre y su alcance constan en `validacion_humana.md`.

Parte C. Simulacro y reconstrucción del incidente

Ejecución técnica comprobada. Contraste automático con Codex y decisión de contención del ejercicio documentada; conformidad de cierre expresada por el usuario con el alcance registrado en validacion_humana.md (validacion_humana.md).

El simulacro se realizó únicamente contra la copia propia de Food Store en PostgreSQL 17.11. La ejecución del cliente abarcó el 22/10/2026 entre 19:45:52.288 y 19:45:52.923 UTC. El log seleccionado corresponde a la sesión `6ada67f0.ee4`, PID 3812, desde `127.0.0.1:60572`, entre 16:45:52.418 y 16:45:52.923 UTC−03.

1. Secuencia diseñada y ejecutada

`sql/simulacro_incidente.sql` define las consultas parametrizadas y `tools/ejecutar_pruebas.py --phase incident` las ejecutó mediante conexiones reales. La conexión de aplicación utilizó `app_web`; otra conexión verificadora obtuvo conteos y membresías sin proporcionar sus resultados a la IA.

1. Se estableció una conexión PostgreSQL válida con SCRAM. Esta autenticación del

motor es anterior y distinta de las llamadas a `fn_autenticar`.

2. Se llamó tres veces a `fn_autenticar` con el mismo usuario comercial sintético

y contraseñas de prueba incorrectas, seguidas de una llamada con la credencial correcta. Los valores se enlazaron con `PQexecParams`; permanecieron fuera del SQL, el log entregado y el repositorio.

3. Se ejecutó realmente `SELECT id,nombre,apellido,eliminado,created_at FROM public.usuario ORDER BY id`.

El cliente recibió 20.001 filas, verificó las cinco columnas y descartó sus valores. No se seleccionó `contrasena` ni se exportaron filas de usuarios.

4. Desde esa misma conexión de `app_web` se intentó

`GRANT admin_datos TO app_web`. La conexión terminó después del rechazo.

La lectura está permitida por las ACL diseñadas para el laboratorio. El ejercicio reproduce una secuencia que requiere investigación en un entorno real; por sí sola no demuestra una vulneración de permisos ni una intención maliciosa.

2. Verificación del intento de escalamiento

El servidor rechazó el GRANT con SQLSTATE **42501**: `app_web` no posee la opción ADMIN sobre `admin_datos`. El evento 16 del log conserva el error y la sentencia asociada; el evento 15 conserva el intento.

La conexión separada `u5_verifier_incident` comprobó que la membresía en `admin_datos` era falsa antes y después. El cliente también confirmó que `session_user` y `current_user` seguían siendo `app_web` tras la autenticación de negocio. Estas comprobaciones están en incident_resultados.json (evidencias/incident_resultados.json): once eventos, estado `passed`. No se presenta un mensaje de éxito de `fn_autenticar` como una elevación del rol PostgreSQL.

3. Extracción y anonimización del log real

Se seleccionó la aplicación exacta `u5_incident` dentro de la ventana UTC del cliente, convirtiendo las marcas del log desde UTC−03. Luego se conservaron **todos** los eventos de su sesión: `line_num` 1 a 17, incluida la conexión inicial, que aún no contiene nombre de aplicación, y la desconexión final.

El archivo entregado es log_simulacro_anonimizado.txt (log_simulacro_anonimizado.txt), con un objeto JSON válido por línea. Se sustituyeron el nombre de la base por `foodstore_lab` y el usuario de las rutas Windows por `USR_LOCAL`; se conservaron horarios, PID, sesión, roles, origen local y códigos de error. Los escaneos de secretos, credenciales y correos y la inspección automática del contenido no detectaron valores personales o contraseñas en el fragmento entregado.

Su SHA-256 es `f30499043574aca7554877e118b1f21f8a804740cdbe2039c32f3f3a2b0d162b`. La trazabilidad de la fuente privada, la selección y las transformaciones está en logs_procedencia.json (evidencias/logs_procedencia.json) y c_anonimizacion_scan.json (evidencias/c_anonimizacion_scan.json). El original privado no integra el repositorio. Esta revisión automática no certifica anonimización irreversible ni sustituye la revisión humana.

4. Consulta real a OpenCode sin contexto adicional

Se consultó OpenCode 1.18.23 con el modelo solicitado `openai/gpt-6.1-sol`, agente `plan` y permisos de herramientas denegados. La consulta se realizó entre 19:49:50.902117 y 19:50:27.880495 UTC. Su único adjunto fue una copia exacta del log anonimizado: no se adjuntaron el guion, los resultados del cliente, los conteos, los controles de membresía ni este informe.

El prompt conservado (ia/c_reconstruccion_prompt.txt) pidió reconstruir la cronología, distinguir hechos de hipótesis y proponer una contención que una persona evaluara antes de actuar. Los metadatos (ia/c_reconstruccion_metadatos.json) registran la consulta real y el hash del único adjunto, idéntico al archivo entregado. OpenCode no ejecutó herramientas ni SQL durante esta consulta.

5. Contraste de la reconstrucción con lo ejecutado

La reconstrucción no presentó errores factuales detectables en el contraste. La IA separó lo visible en el log de las hipótesis y las comprobaciones externas; la tabla siguiente identifica sus aciertos y las cuestiones que únicamente resuelve la evidencia complementaria del cliente.

Reconstrucción de OpenCode	Contraste con la evidencia y el cliente	Evaluación
Una sesión de `app_web`, origen local, SCRAM y aplicación declarada `u5_incident`.	Eventos 1–4; el cliente confirma rol de sesión y rol efectivo `app_web`.	Correcto; no atribuye una persona al rol.
Tres resultados `success=f` y luego uno `success=t`.	Eventos 6, 8, 10 y 12; el cliente obtiene NULL tres veces y el identificador esperado en la cuarta.	Correcto; la IA no puede saber por el log si se usó la misma cuenta. El guion sí lo confirma.
El SELECT solicita cinco columnas y no tiene WHERE; no prueba entrega ni cantidad de filas.	Evento 14; el cliente recibió y descartó 20.001 filas.	Correcto. No mencionar 20.001 es un límite del registro, no una omisión errónea.
La segunda consulta de identidades no muestra sus resultados.	Evento 13; el cliente confirma que los roles permanecieron iguales.	Correcto; la comprobación externa complementa el log.
GRANT rechazado con 42501; el fragmento no revela membresías anteriores.	Eventos 15–16; el verificador independiente obtuvo membresía falsa antes y después.	Correcto. Ese estado no fue adjuntado a la IA.
No demuestra fuerza bruta, exfiltración, compromiso ni intención maliciosa.	El guion era un laboratorio autorizado; no hubo exportación de filas ni contención automática.	Lectura prudente y coherente con el alcance disponible.

Reconstrucción de OpenCode	Contraste con la evidencia y el cliente	Evaluación
La conexión ya terminó y no corresponde terminar esa sesión cerrada.	Evento 17: duración informada de 0,511 segundos.	Correcto; la propuesta de restringir futuras conexiones es condicional.

La hipótesis de exploración de datos e intento de obtener membresía es compatible con la secuencia, pero no establece su intención. Tampoco se puede inferir un origen externo a partir de `127.0.0.1`. Las correcciones reales de otras consultas de IA se documentan donde ocurrieron; no se atribuye a esta reconstrucción un error inexistente.

6. Decisión de contención del ejercicio

La decisión del ejercicio es preservar la evidencia, verificar el alcance y condicionar cualquier restricción a la confirmación de un incidente y a la evaluación de impacto; ninguna medida de contención fue ejecutada. Esta conclusión se presentó al usuario junto con la explicación de A–E, tras lo cual pidió «bien entonces enviame el zip listo para entregar asi lo subo a la plataforma». Su pedido expresa conformidad con el cierre de la entrega, sin establecer una revisión exhaustiva del PDF ni una corrección técnica humana; el alcance se conserva en validacion_humana.md (validacion_humana.md).

En un caso real, primero se preservarían los logs con acceso restringido y se verificarían la autorización de la actividad, el componente que usa `app_web`, el alcance de la lectura y el estado actual de sus privilegios y sesiones. La conservación de evidencia precede a cambios que puedan dificultar reconstruir lo ocurrido.

Solo si esa revisión confirma un incidente, la persona responsable evaluaría suspender temporalmente la credencial de `app_web` y sus sesiones activas relacionadas, considerando el impacto sobre los servicios. Se rotarían únicamente los secretos comprometidos o cuya confidencialidad ya no pueda sostenerse, con revisión posterior de acceso y monitoreo.

No se propone bloquear `127.0.0.1` como si fuera un atacante remoto ni revocar una membresía en `admin_datos` que las pruebas comprobaron inexistente. La sesión del fragmento ya estaba cerrada. El apagado posterior del servidor privado fue un cierre rutinario del laboratorio realizado por su operador, separado de la respuesta de OpenCode; no constituye ejecución de una contención sugerida por IA.

Apéndice. Reconstrucción completa y evidencia enlazada

La respuesta íntegra recibida de OpenCode (ia/c_reconstruccion_respuesta.md) se entrega sin modificaciones en el repositorio; este informe solo la contrasta. Se conservan también el prompt (ia/c_reconstruccion_prompt.txt), los metadatos originales (ia/c_reconstruccion_metadatos.json) y la copia exacta del único adjunto enviado (ia/c_reconstruccion_adjuntos/01_log_simulacro_anonimizado.txt). El resultado del cliente (evidencias/incident_resultados.json) y la procedencia de los logs (evidencias/logs_procedencia.json) permiten reproducir la validación externa y distinguirla del material que efectivamente vio la IA.

Parte D - Protección de datos y equivalencia de agregados

Food Store - Unidad 5 - Base de Datos II. Exequiel Arancibia y Joaquin Romero.

La reconstrucción final, posterior a la Parte A y registrada el **22/10/2026 a las 16:46:53, zona UTC-03**, construyó una copia de **20.001 usuarios** y verificó **dos grupos**, con **cero diferencias en ambos sentidos**. Las evidencias actuales son d_ejecucion_psql.txt (d_ejecucion_psql.txt), d_equivalencia.json (d_equivalencia.json) y d_resultados_lado_a_lado.md (d_resultados_lado_a_lado.md); el SQL completo está en usuario_anon.sql (../sql/usuario_anon.sql). La tabla resultante pertenece a `rol_propietario` y `rol_reportes` tiene SELECT efectivo comprobado.

El SQL de construcción y comparación fue propuesto en una consulta real a **OpenCode 1.18.23**, con el modelo seleccionado explícitamente `openai/gpt-6.1-sol`, a partir de un contexto de esquema sanitizado, sin registros de usuarios. El prompt, la respuesta completa y los metadatos se conservan en `ia/d_prompt.txt`, `ia/d_respuesta.md` e `ia/d_metadatos.json`. Después de A, Codex integró `SET LOCAL ROLE rol_propietario` y el uso del esquema ya configurado; esa adaptación técnica no se atribuye a la respuesta original de OpenCode. La persona solicitante dio conformidad al cierre y pidió el ZIP para subirlo a la plataforma después de recibir el resumen A–E y la propuesta de revisión. El alcance consta en validacion_humana.md (validacion_humana.md): esa conformidad no acredita lectura exhaustiva ni validación individual de ambos integrantes, y los controles automáticos conservan su autoría técnica.

1. Construcción de la copia independiente

Se creó `datos_anon.usuario_anon` con exactamente seis columnas: `folio`, `nombre`, `apellido`, `email`, `mes_alta` y `eliminado`. La copia no conserva una columna de ID original, la fecha y hora exactas de alta, contraseñas, hashes ni teléfonos reales; no modifica `public.usuario`.

Durante la construcción se obtiene `row_number() OVER (ORDER BY id)` y se usa su resultado para formar nombres del tipo `Persona00001`, el apellido constante `Laboratorio` y correos del tipo `persona00001@example.invalid`. La longitud del número es `GREATEST(5, length(folio::text))`, por lo que no se trunca cuando supera cinco dígitos; el ID de origen se usa únicamente para ordenar esa construcción y no se persiste en una tabla de correspondencias.

El mes conservado se obtiene con esta expresión, que respeta la zona del negocio antes de agrupar:

```
date_trunc(
    'month',
    created_at AT TIME ZONE 'America/Argentina/Buenos_Aires'
)::date
```

La reconstrucción de la copia y sus verificaciones se ejecutan dentro de `BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ` y `COMMIT`. Esto conserva el mismo snapshot de la fuente durante toda la transacción; si se detectan diferencias, la afirmación del script lanza una excepción y evita confirmar la reconstrucción. El reemplazo se limita a la copia de laboratorio, sin `CASCADE` y dentro de esa transacción.

La conexión técnica de preparación ejecuta el SQL, pero dentro de la transacción cambia al creador previsto mediante `SET LOCAL ROLE rol_propietario`; no se otorgó membresía de ese propietario a los logins de aplicación. La ejecución final registra `modo_ejecucion = propietario`, `propietario_tabla = rol_propietario` y `rol_reportes_select = true`. Así se comprobó la reconstrucción bajo el creador cuyos defaults permiten SELECT a reportes en `datos_anon`, además de la migración explícita de la copia preexistente que ya había realizado A.

La evidencia registra la misma SHA-256 para el archivo fuente y los bytes efectivos enviados a psql, sin una sustitución de rol en el modo normal:

```
7e49e29525feecb9914cd8d540dfa32b8f2bef53c7fd6f98464f0158a51150a9
```

Dos momentos reales y reproducción

La ejecución inicial del **21/10/2026 a las 13:54:59, UTC-03**, anterior a A, se conserva por separado en d_bootstrap_equivalencia.json (evidencias/d_bootstrap_equivalencia.json) y d_bootstrap_ejecucion_psql.txt (evidencias/d_bootstrap_ejecucion_psql.txt). También obtuvo 20.001 usuarios, dos grupos y diferencias 0/0; no se la presenta como una prueba de permisos de roles finales todavía inexistentes.

Para reproducir ese orden desde un laboratorio recién preparado:

1. Ejecutar `python tools/ejecutar_anonimizacion.py --bootstrap` únicamente en la preparación previa a A. Ese modo explícito sustituye la única línea de cambio de rol por la creación del esquema de copia y registra el SQL efectivo utilizado.

2. Aplicar la Parte A, con su migración de propietario y REVOKE/GRANT explícitos sobre la copia ya existente. Los defaults afectan objetos futuros del creador configurado, no corrigen retroactivamente una tabla anterior.

3. Ejecutar `python tools/ejecutar_anonimizacion.py` sin `--bootstrap`. Este es el modo final realizado: reconstruye bajo `rol_propietario`, vuelve a comparar el contenido agregado y verifica propietario y permiso SELECT de reportes.

Si se conservan ambos momentos en una reproducción nueva, deben archivarse las evidencias iniciales antes de ejecutar el modo normal, que escribe los archivos actuales `d_*`. El rol de reportes es NOLOGIN: la comprobación de su privilegio no implica que se haya iniciado sesión directamente con él.

2. Consultas equivalentes solicitadas a OpenCode

`usuario` no tiene un atributo de rol de negocio: los roles LOGIN de PostgreSQL son identidades diferentes. Por eso la comparación utiliza **mes local de alta y marca de eliminación**, sin inventar una columna `rol`.

Consulta sobre la tabla real, cuyo resultado solo contiene agregados:

```
SELECT
    date_trunc(
        'month',
        created_at AT TIME ZONE 'America/Argentina/Buenos_Aires'
    )::date AS mes_alta,
    eliminado,
    count(*) AS cantidad
FROM public.usuario
GROUP BY
    date_trunc(
        'month',
        created_at AT TIME ZONE 'America/Argentina/Buenos_Aires'
    )::date,
    eliminado
ORDER BY mes_alta, eliminado;
```

Consulta equivalente sobre la copia:

```
SELECT mes_alta, eliminado, count(*) AS cantidad
FROM datos_anon.usuario_anon
GROUP BY mes_alta, eliminado
ORDER BY mes_alta, eliminado;
```

A OpenCode se le proporcionó únicamente el esquema sanitizado para generar estas consultas. Las salidas conservadas después de ejecutarlas contienen agregados; para una revisión posterior con IA se limita la información a esos agregados revisados, sin enviar filas reales ni la tabla de usuarios completa.

3. Resultados reales y comprobación de equivalencia

Mes local de alta	Eliminado	Cantidad en usuario real	Cantidad en usuario_anon	Coincide
2026-09-01	false	20.000	20.000	Sí
2026-10-01	false	1	1	Sí
Total		**20.001**	**20.001**	**Sí**

En la salida original de `psql`, `f` representa `false`. Las dos consultas devuelven las mismas filas, agrupamientos y conteos; la comparación lado a lado mediante `FULL OUTER JOIN` también marca `coincide = t` en ambos grupos.

La auditoría aplica `EXCEPT ALL` en ambos sentidos para detectar tanto grupos o conteos faltantes como sobrantes. Una versión autocontenida de ese control, equivalente al ejecutado, es:

```
WITH origen AS (
    SELECT
        date_trunc(
            'month',
            created_at AT TIME ZONE 'America/Argentina/Buenos_Aires'
        )::date AS mes_alta,
        eliminado,
        count(*) AS cantidad
    FROM public.usuario
    GROUP BY 1, 2
), copia AS (
    SELECT mes_alta, eliminado, count(*) AS cantidad
    FROM datos_anon.usuario_anon
    GROUP BY 1, 2
)
SELECT 'origen_menos_copia' AS sentido, count(*) AS diferencias
FROM (TABLE origen EXCEPT ALL TABLE copia) d
UNION ALL
SELECT 'copia_menos_origen', count(*)
FROM (TABLE copia EXCEPT ALL TABLE origen) d;
```

Sentido de la comparación	Diferencias reales
origen_menos_copia	0
copia_menos_origen	0

En el script final ejecutado, las diferencias se almacenan temporalmente y se muestran: la salida fue `(0 rows)`. El bloque `DO` comprueba su inexistencia y lanza `RAISE EXCEPTION` si encuentra alguna; el registro muestra `BEGIN`, `SET`, `INSERT 0 20001`, `DO` y luego `COMMIT`, sin error. La tabla temporal se elimina al confirmar y no forma parte de la copia entregada.

Además, el verificador contrastó la huella de los **20.000 usuarios heredados** y confirmó que permanecen intactos; el usuario adicional corresponde a la cuenta del laboratorio U5. El control final más amplio de conservacion_after.json (evidencias/conservacion_after.json), comparado con conservacion_before.json (evidencias/conservacion_before.json), registra `same_as_before = true` para conteos y huellas ordenadas de las cinco tablas de negocio, incluidas las 20.001 filas actuales de usuario. Esto comprueba conservación del contenido observado, por separado de la equivalencia de agregados y de los permisos.

Alcance y límites de la transformación

El correlativo es determinista para el mismo snapshot y orden por ID, pero no constituye un identificador estable frente a cualquier modificación de la fuente: las altas pueden agregar nuevas posiciones y los borrados pueden desplazar las posteriores. Si los IDs de origen forman un rango denso que comienza en uno, el folio puede incluso coincidir numéricamente con el ID; quitar una columna llamada `id` no elimina esa posibilidad de vinculación.

Por eso esta técnica se trata como una copia desidentificada de laboratorio, con posibles propiedades de pseudonimización, y **no como una demostración de anonimización irreversible**. `mes_alta` y `eliminado` siguen siendo cuasiidentificadores; un grupo de una sola persona, como el de octubre en esta prueba, puede facilitar inferencias al combinarlo con información externa. La igualdad comprobada acredita solamente los agregados definidos, no ausencia de reidentificación ni equivalencia de cualquier consulta posible sobre los datos.

4. Información que no debe salir hacia un asistente de IA

Las contraseñas, hashes de autenticación y credenciales de acceso de Food Store no deben enviarse a un asistente de IA. Tampoco deben salir nombres, correos, teléfonos ni ventas vinculadas a una persona sin una transformación y minimización adecuadas al uso previsto. Para trabajar con IA se prefieren esquemas sin registros, datos sintéticos o agregados revisados, evaluando también el riesgo de grupos pequeños y cuasiidentificadores.

Bitácora de uso de IA — Unidad 5

Integrantes: Exequiel Arancibia y Joaquin Romero. Las pruebas técnicas están completas; la conformidad de cierre de la persona solicitante y su alcance se documentan en `docs/validacion_humana.md`. No se atribuye la revisión automática de Codex a los integrantes.

Se utilizó Kiro real y OpenCode 1.18.23, modelo `openai/gpt-6.1-sol`, en modo plan sin herramientas. Los prompts y respuestas completas se conservan, junto con metadatos y huellas de integridad. Kiro generó contenido en su chat; Codex guardó copias fieles desde Export Chat, sin presentarlas como escrituras nativas de Kiro.

Prompt enviado a la IA	Respuesta relevante recibida (resumida)	Validación o corrección humana aplicada
A, Kiro: `docs/kiro_roles_spec/prompt_inicial.txt`, contexto y prompts de continuación/revisión/corrección/diseño de esa carpeta. Planificar roles y matriz antes de escribir SQL.	Generó requirements y design reales. El borrador añadió SELECT de pedido/detalle a soporte y presentó una contradicción de propietarios; se conservan versiones anteriores y correcciones. El texto inicial también tenía una frase ambigua sobre propiedad, que se aclaró.	Cierre solicitado por la persona usuaria; ver registro de su alcance. Codex detectó la lectura extra comparando la matriz de contacto y pidió quitarla; Kiro corrigió su respuesta. Se adoptan seis columnas finales de soporte, sin ampliar la prueba a siete. Los contratos y precisiones posteriores de Codex se distinguen del texto de Kiro.
A, OpenCode: `docs/ia/a_revision_prompt.txt`, auditar roles.sql y comparar alternativas operativas que simplificaran mantenimiento.	No encontró incumplimiento bloqueante demostrado. Advirtió sobre literales en logs, ACL ajenas y canal temporal de autenticación. A solicitud explícita, propuso como alternativas heredar lectura para soporte o GRANT SELECT ON ALL TABLES de public para reportes, señalando que excedían la matriz.	Cierre solicitado por la persona usuaria; ver registro de su alcance. Codex rechazó ambas ampliaciones: soporte recibiría pedidos/catálogo y reportes llegaría a contrasena y datos reales. Se conservaron grants puntuales. Son candidatas SOLICITADAS y advertidas por el auditor; no se presentan como errores espontáneos ni como medidas aplicadas. La comparación real dio 44 ACL, faltantes 0 y adicionales 0.
B, OpenCode: `docs/ia/bc_prompt.txt`, generar y revisar catálogo de auditoría con parámetros sobre el esquema sanitizado.	Propuso lectura, pedido, detalle, actualización y ROLLBACK; exigió verificar filas, estado PENDIENTE y configuración de parámetros. Su SELECT no incorporaba el filtro de producto apto que recomendaba.	Cierre solicitado por la persona usuaria; ver registro de su alcance. Codex añadió eliminado=FALSE AND stock>=2, precio de catálogo, RETURNING y comprobación efectiva de cardinalidad; ajustó también el orden real del cliente para BEGIN antes de la lectura. La ejecución aprobó 10 eventos, mantuvo las filas por ROLLBACK y capturó FATAL 28P01 real para el login fallido.
C, OpenCode: mismo `bc_prompt.txt`, generar script de autenticaciones, lectura masiva y escalamiento rechazado.	Entregó consultas parametrizadas y recomendó comprobar membresía antes/después, distinguir identidades y no confundir una consulta registrada con entrega de filas.	Cierre solicitado por la persona usuaria; ver registro de su alcance. Codex ejecutó tres fallos y un éxito comercial, verificó rol SQL sin cambio, recibió y descartó 20.001 filas y confirmó GRANT rechazado con 42501, con membresía false antes/después. El comentario de IA sobre reutilizar la consulta de membresía se trató como mejora documental: el cliente ya comprobaba ambos estados.

Prompt enviado a la IA	Respuesta relevante recibida (resumida)	Validación o corrección humana aplicada
C, OpenCode: `docs/ia/c_reconstruccion_prompt.txt`, reconstruir exclusivamente el log anonimizado y proponer contención condicionada a evaluación de una persona.	Reconstruyó las 17 entradas, actor app_web, cuatro resultados de la función, SELECT, GRANT denegado y desconexión. Separó autenticación del motor, resultado comercial e hipótesis; no afirmó fuerza bruta, exfiltración ni membresía previa. Propuso evaluar restricción temporal sólo si se confirma actividad no autorizada.	Decisión de contención documentada para el ejercicio y cierre solicitado por la persona usuaria; ver registro. El contraste automático encontró correspondencia con el log y explicó las omisiones que éste no podía resolver: 20.001 filas y membresía false provienen del cliente, no del fragmento. No se inventa un error de la IA. Se propone preservar evidencia y verificar autorización antes de contener, sin ejecutar acciones automáticamente ni bloquear loopback como si fuera un atacante externo.
D, OpenCode: `docs/ia/d_prompt.txt`, construir copia independiente determinista y consultas equivalentes por mes local/eliminado; comparar ambos sentidos con EXCEPT ALL.	Generó SQL REPEATABLE READ, seis columnas sintéticas, agregados equivalentes y afirmación ejecutable; advirtió sobre correlación y reidentificación.	Cierre solicitado por la persona usuaria; ver registro de su alcance. Codex ejecutó la versión inicial bootstrap y luego integró SET LOCAL ROLE rol_propietario, conservando la respuesta original. La ejecución final verificó 20.001 filas, dos grupos, diferencias 0/0, propietario correcto y SELECT de reportes. No se afirma anonimización irreversible: el correlativo puede coincidir numéricamente con un ID denso y los grupos pequeños permiten inferencias.
E, solicitud del trabajo completo con explicación y entrega; consolidación asistida por Codex.	Codex organizó informes, tabla obligatoria, referencias de procedencia, historial y generadores del PDF/ZIP. No se realizó una consulta adicional de OpenCode para E ni se simula una respuesta de esa herramienta.	Cierre solicitado por la persona usuaria; ver registro de su alcance. Los resultados técnicos se vinculan a archivos de ejecución y los prompts reales permanecen completos. El paquete conserva Git; los secretos, respaldos y logs originales quedan excluidos.

Caso más significativo de imprecisión detectada

El permiso adicional de lectura de pedidos y detalles propuesto por Kiro para soporte excedía la necesidad de consultar contactos y actualizar teléfono. La comparación con esa finalidad permitió detectar la ampliación, rechazarla técnicamente y pedir una corrección real; los borradores y el prompt de revisión acreditan el proceso. La contradicción inicial de propietarios se registra con su causa compartida: el contexto tenía una frase ambigua que debió precisarse. La conformidad de cierre se registra con las palabras de la persona solicitante, sin convertir esta revisión automática en una corrección humana ni atribuir lectura exhaustiva.

La auditoría OpenCode de A aporta además dos alternativas más amplias concretas que no se aplicaron. El propio modelo explicó su riesgo después de que se le pidieran alternativas de mantenimiento; reconocer esa advertencia es parte de la lectura crítica. El caso no se transforma retrospectivamente en una recomendación espontánea insegura.

Reconstrucción de artefactos de clase

Los antecedentes no contenían el esquema de roles ni fn_autenticar/fn_resetear_password de las clases U5. Se reconstruyeron sobre las tablas existentes al implementar A, después de la planificación previa de Kiro. No se afirma que se recuperaron archivos de clase. Las funciones incorporan bcrypt, permisos internos puntuales, retorno id/NULL o void según su contrato, nombres calificados y registro sin credenciales.

Límites y validación

Una lectura masiva permitida por las ACL demuestra la superficie del rol técnico; no acredita intención maliciosa ni exfiltración. Los tres fallos de fn_autenticar son resultados comerciales en una sesión PostgreSQL ya autorizada y no son errores 28P01. La decisión de contención del ejercicio y la conformidad de cierre se registran en el informe C y en el registro de revisión; no se deriva automáticamente de una respuesta de IA.

Los intentos operativos fallidos se conservan como tales: primero no pudo habilitarse Kiro; luego el usuario rechazó manualmente su solicitud de terminal y se continuó sólo por chat. La consulta D inicial falló por el sandbox antes de un intento exitoso; un proveedor predeterminado de OpenCode no tuvo cuota y se utilizó el modelo disponible registrado. No se convierten fallos de transporte en respuestas del asistente.

Historial Git y mejora del proceso

El historial conserva commits reales por preparación, D, B/C preliminar, planificación Kiro, implementación A y evidencias posteriores. El commit `319cbaa` guarda la especificación real; `4262d81` incorpora después roles.sql y `8f8d3f8` su verificación. Se usó la identidad técnica Codex, sin suplantar a los integrantes ni reescribir fechas.

D se avanzó antes de A mientras Kiro estaba bloqueado; por eso el historial no sigue estrictamente el orden A–E. En una próxima iteración convendría asegurar primero el acceso a todas las herramientas y separar desde el inicio el bootstrap de la reconstrucción bajo el propietario definitivo. Los archivos de metadatos mantienen hashes de las versiones realmente consultadas; las correcciones posteriores no sustituyen silenciosamente esos adjuntos.

Las diferencias entre borradores, versiones finales y consultas se explican en `docs/ia/procedencia_y_versiones.md`. No se subió el repositorio a un hosting ni se envió el trabajo a terceros.

Conformidad de cierre y alcance de la revisión humana

El 22/10/2026 se presentó a la persona solicitante el PDF completo y la propuesta de conservar permisos puntuales, rechazar alternativas amplias y, ante un incidente real confirmado, preservar evidencia y evaluar restringir app_web con intervención responsable, sin contención automática.

La persona preguntó si el resultado estaba dentro de los parámetros y las consignas. Se le explicó la cobertura de A–E y que la entrega requerida es el ZIP del repositorio con Git, acompañado por el PDF. Su respuesta final fue:

> bien entonces enviame el zip listo para entregar asi lo subo a la plataforma

Esta respuesta expresa conformidad para cerrar y entregar el trabajo tras la explicación y la propuesta presentadas. Se registra ese alcance concreto: no constituye una declaración de haber leído exhaustivamente cada archivo ni una aprobación individual documentada de ambos integrantes.

Las correcciones técnicas fueron realizadas con Codex; no se atribuyen a personas. La decisión de contención del ejercicio queda documentada en el informe C. Ninguna medida fue ejecutada automáticamente ni se envió el trabajo a terceros: el archivo se entrega a la persona para que lo suba.

Las menciones a validación pendiente dentro de respuestas y metadatos originales de IA reflejan el estado al realizar esas consultas y se conservan como parte de su procedencia. No se reescriben para aparentar aprobaciones anteriores a esta conversación.

Food Store — Trabajo práctico Unidad 5

Integrantes: Exequiel Arancibia y Joaquin Romero. **Asignatura:** Base de Datos II — UTN, ciclo 2026. **Motor verificado:** PostgreSQL 17.11 sobre Windows.

La resolución continúa los TP1 a TP5 y la Unidad 4. El ZIP contiene el repositorio y su carpeta `.git`, con commits progresivos; el PDF reúne las explicaciones paso a paso. Las pruebas técnicas A–D están completas. La persona solicitante dio conformidad al cierre mediante su pedido final de entrega; su respuesta y el alcance de esa conformidad se registran en `docs/validacion_humana.md`.

Archivos principales

Parte	Resolución y evidencia
A	`sql/roles.sql`; `docs/kiro_roles_spec/requirements.md` y `design.md`; `docs/informe_roles.md`; salidas reales de `\du`, ACL y comparación exacta en `docs/evidencias/a_*`.
B	`sql/configurar_auditoria.sql`, `sql/auditoria_test.sql`; `docs/informe_auditoria.md`; `docs/log_auditoria_prueba.txt`.
C	`sql/simulacro_incidente.sql`; `docs/log_simulacro_anonimizado.txt`; respuesta completa `docs/ia/c_reconstruccion_respuesta.md`; `docs/informe_incidente.md`.
D	`sql/usuario_anon.sql`; consultas real/copia y resultados lado a lado en `docs/verificacion_equivalencia.md`; resultados originales `docs/d_ejecucion_psql.txt` y `d_equivalencia.json`.
E	`BITACORA.md`; prompts, respuestas y procedencia en `docs/ia/`; historial Git completo.

El informe complementario es `docs/TP_U5_FoodStore_Arancibia_Romero.pdf`. Los SQL B/C son catálogos de consultas parametrizadas: se ejecutan mediante el orquestador Python y sus marcadores, no enviando el archivo entero a psql.

Resultados comprobados

Comprobación real	Resultado
Roles y permisos existentes	Ocho roles; 44 entradas ACL de negocio exactas, sin faltantes ni adicionales; tres membresías INHERIT TRUE, SET FALSE, ADMIN FALSE.
Permisos y reset	52 eventos de verificación aprobados, con conexiones reales app_web/admin_datos y pruebas de grupos claramente identificadas.
Objetos futuros	71 eventos aprobados; public sin acceso automático de negocio; datos_anon con SELECT de reportes; funciones sin EXECUTE público para tres creadores. Probes revertidos.
Auditoría	Lectura, inserción, actualización y login PostgreSQL fallido reales; filas de prueba revertidas.
Simulacro	Tres fallos comerciales y un éxito; 20.001 filas recibidas y descartadas; escalamiento rechazado con 42501; membresía administrativa false antes y después.
Equivalencia	20.001 filas por copia; dos grupos; diferencias EXCEPT ALL 0/0; propietario rol_propietario y SELECT de reportes verificados.
Conservación	Conteos y huellas ordenadas de las cinco tablas de negocio iguales antes/después.

Los 20.000 usuarios heredados permanecen intactos; una cuenta sintética adicional sirve para autenticación. Se conservan los índices, vistas, reglas TP2, extensión FNBC y materializadas anteriores. La lectura masiva no equivale por sí sola a exfiltración ni identifica a una persona; los permisos por columna no implementan aislamiento por fila.

Reproducción desde una copia U4

Se requieren Windows, Python y PostgreSQL 17 instalado. La generación del PDF requiere además ReportLab y las fuentes Arial/Consolas de Windows. El operador debe aportar privadamente un respaldo equivalente de su Food Store acumulativo U4: se excluye del ZIP porque contiene datos y credenciales almacenadas. Los hashes del respaldo utilizado y la continuidad están en `docs/continuidad_y_alcance.md`.

Ejecutar desde la raíz del repositorio en PowerShell. Este bloque crea un laboratorio NUEVO; el preparador rechaza un runtime existente y no borra otro servidor.

```
$respaldoU5 = 'C:\ruta-privada\base_u4.dump'
$repoU5 = (Get-Location).Path
$runtimeU5 = Join-Path (Split-Path -Parent $repoU5) 'food-store-tp-u5-runtime'
python .\tools\preparar_laboratorio.py --source-dump $respaldoU5 --runtime $runtimeU5 --port 55605
$env:U5_SECRET_FILE = Join-Path $runtimeU5 'secrets.json'
python .\tools\crear_cuenta_prueba.py
python .\tools\ejecutar_anonimizacion.py --bootstrap
python .\tools\verificar_conservacion.py --phase before --evidence-dir (Join-Path $runtimeU5
  'evidencias')
python .\tools\instalar_seguridad.py
python .\tools\ejecutar_pruebas.py --phase permissions
python .\tools\ejecutar_pruebas.py --phase defaults
python .\tools\ejecutar_anonimizacion.py
python .\tools\verificar_matriz.py
python .\tools\capturar_permisos.py
python .\tools\activar_auditoria.py
```

```
python .\tools\ejecutar_pruebas.py --phase audit
python .\tools\ejecutar_pruebas.py --phase incident
python .\tools\verificar_conservacion.py --phase after --evidence-dir (Join-Path $runtimeU5
  'evidencias')
python .\tools\rotar_log.py
```

Primero D con `--bootstrap` crea la copia previa exigida por la migración A; después D sin esa opción la reconstruye realmente bajo `rol_propietario` y comprueba sus permisos. La adaptación de bootstrap sustituye únicamente la línea SET LOCAL ROLE por CREATE SCHEMA IF NOT EXISTS y registra el SHA-256 del SQL efectivo. La cuenta y los roles se instalan antes de habilitar sentencias en el log; las contraseñas viajan como parámetros y no aparecen en los archivos entregados.

Las especificaciones Kiro conservadas se generaron realmente antes de roles.sql. No hace falta regenerarlas para reproducir el SQL. Si se cambia la política, primero se revisan las matrices y luego el SQL: no se amplían permisos sólo para hacer pasar una prueba.

Captura y revisión de logs

El JSON nativo permanece en el runtime privado. Después de rotar, se seleccionan las sesiones completas de las ventanas UTC de `audit_resultados.json` e `incident_resultados.json`, correlacionando la zona efectiva de `configuracion_auditoria.json`; el FATAL 28P01 del login fallido se identifica además por rol, instante y session_id.

`tools/anonimizar_log.py` recibe el fragmento privado seleccionado y escribe la copia pública; admite `--application` y `--session`. El procedimiento y las sesiones reales de esta ejecución se registran en `docs/evidencias/logs_procedencia.json`. Se revisa la anonimización antes de consultar a una IA. OpenCode recibió sólo el log C anonimizado y el prompt neutral conservado; no recibió la secuencia del cliente ni los conteos verificados por separado.

Los logs registran sentencias e intentos, incluidos cambios revertidos. No muestran valores OLD/NEW de cada fila ni prueban por sí solos cuántas filas recibió el cliente. La decisión de contención descrita es humana e hipotética; no se ejecutó automáticamente ninguna medida sobre cuentas, red o roles.

Seguridad del paquete y herramientas de IA

El repositorio excluye respaldos, runtime, claves, hashes de contraseñas y logs originales. Los secretos aleatorios se guardan fuera de Git con ACL Windows restringida al usuario actual, SYSTEM y Administradores. El servidor de pruebas escucha sólo en loopback y usa SCRAM. No se publican filas de usuario ni parámetros de autenticación.

Se usaron Kiro real y OpenCode 1.18.23 con el modelo `openai/gpt-6.1-sol`; las respuestas completas se conservan sin alterar sus bytes. Codex implementó herramientas, integró propuestas y ejecutó verificaciones: esa asistencia automática se distingue de la revisión humana. Los commits usan la identidad técnica Codex y no suplantan a los integrantes.

Para generar el PDF y verificar el paquete, tras guardar y comprometer los informes finales:

```
python .\tools\generar_informe_pdf.py
git add docs/TP_U5_FoodStore_Arancibia_Romero.pdf
git commit -m 'docs: actualizar informe PDF'
python .\tools\empaquetar_entrega.py --dry-run
python .\tools\empaquetar_entrega.py
git log --oneline --reverse
```

El empaquetador exige árbol Git limpio, verifica hashes del manifiesto y comprueba el repositorio restaurado. La entrega prevista por esta guía es el ZIP con Git o un enlace al repositorio; el PDF acompaña esa entrega. El trabajo se deja preparado para que los integrantes lo presenten por el canal del aula virtual.